Git & GitLab Professional – Tag 2

Handout: GitLab im Team · Gruppen & Rollen · Zugänge · Planung · Merge Requests · CI/CD · Container · Runner

5 GitLab, Gruppen & Rollen

Git ist die Werkzeugkiste, **GitLab** die ganze Werkstatt: Planen → Code → Review → Testen → Ausliefern. Betrieb als **gitlab.com** (Cloud) oder selbst betrieben; Tarife Free · Premium · Ultimate.

- **Gruppe:** Team oder Abteilung · **Subgruppe:** Unterordner, z. B. `backend`
- **Projekt:** Repository plus Issues, Pipelines, Einstellungen
- **Namespace:** der Pfad in der URL – `gitlab.com/gruppe/backend/calc-service`
- **Vererbung:** Mitglied der Gruppe = Mitglied in allen Projekten darunter
- **Benutzer** = Konto · **Mitglied** = Benutzer mit Rolle (direkt oder geerbt)
- Einladungen **mit Ablaufdatum** – ideal für Externe

Rolle	darf (zusätzlich zur Stufe darunter)
10 Guest	Issues sehen und anlegen
20 Reporter	Code lesen und klonen, Issues verwalten – **nicht pushen**
30 Developer	Branches pushen, Merge Requests erstellen
40 Maintainer	`main` schützen, mergen, Projekteinstellungen
50 Owner	Gruppe verwalten und löschen

Least Privilege: so viel Recht wie nötig, so wenig wie möglich.

5 SSH-Keys, Projekte & Projekteinstellungen

- **Privater Schlüssel** bleibt auf Ihrem Rechner – nie weitergeben
- **Öffentlicher Schlüssel** (`.pub`) wird in GitLab hinterlegt
- **Passphrase** schützt den Schlüssel bei Laptop-Verlust
- Ein Schlüssel pro Gerät, mit Ablaufdatum

```
ssh-keygen -t ed25519 -C "max@firma.de"
ssh-keygen -p -f ~/.ssh/id_ed25519  # Passphrase
ssh -T git@gitlab.com  # → "Welcome to GitLab"
```

Firmennetz sperrt Port 22? `altssh.gitlab.com` über Port 443 nutzen.

Projekte anlegen

- Leer, aus einer Vorlage oder per Import (z. B. von GitHub)
- **Immer in einer Gruppe**, nie im persönlichen Namespace
- README, Beschreibung und Topics helfen beim Finden
- **Archivieren statt löschen:** schreibgeschützt, Historie bleibt

Zentrale Einstellungen

Schalter	Wirkung
Protected Branch	niemand pusht direkt auf `main` – nur per Merge Request
Pipelines must succeed	rote Tests blockieren den Merge
Approvals	Vier-Augen-Prinzip vor dem Merge
Merge-Methode	Merge-Commit · Semi-linear · Fast-forward
Delete source branch	Branch nach dem Merge automatisch löschen

Wo? *Settings → Repository → Protected branches* und *Settings → Merge requests*.

Push auf main abgelehnt? Commit auf einen Branch retten: `git switch -c fix/x` → `git switch main` → `git reset --hard origin/main` → Branch pushen.

6 Sichtbarkeit, Tokens & Keys

Sichtbarkeit	Wer sieht es?
Private	nur Mitglieder – Standard für Firmencode
Internal	alle angemeldeten Nutzer der Instanz (nicht auf gitlab.com)
Public	alle, auch ohne Login

Teilen mit einer anderen Gruppe: deren Mitglieder bekommen höchstens die gewählte Rolle.

Zugang	für …
SSH-Key · Personal Access Token	Person – eigene Skripte, API
Project-/Group-Token	Bot, Integrationen
Deploy Token	Server holt Code/Images per HTTPS
Deploy Key	Server-SSH-Schlüssel für **ein** Projekt
`CI_JOB_TOKEN`	jeder Pipeline-Job, automatisch

Access Tokens richtig nutzen

```
read -rs GITLAB_TOKEN && export GITLAB_TOKEN
# Eingabe unsichtbar, nicht in der History
curl -s -H "PRIVATE-TOKEN: $GITLAB_TOKEN" \
  "https://gitlab.com/api/v4/projects?
membership=true&simple=true"
```

- **Scope** so klein wie möglich: `read_api` statt `api`
- **Immer mit Ablaufdatum**
- Nie in Code, Chat, Ticket oder Shell-History – geleakt? Sofort widerrufen

Deploy Token & Deploy Key gehören dem Projekt, nicht einer Person: Standard nur lesend, überleben Urlaub und Kündigung. Persönliche Tokens werden mit dem Konto ungültig – deshalb nie für nächtliche Deployments.

7 Planung: Issues, Boards & Milestones

- **Issue:** Titel, Beschreibung, Verantwortliche, Termin, Diskussion
- **Tasks:** Checkliste im Issue mit `- [ ] …`
- **Labels** sortieren; `status::doing` ersetzt `status::todo` automatisch (Scoped Labels, Premium)
- **Board:** Spalten = Labels; Karte verschieben = Label ändern, nach „Closed“ = Issue schließen

```
/assign @name       # zuweisen
/label ~prio::high  # Label setzen
/estimate 3h        # Schätzung
/spend 1h30m        # gearbeitete Zeit
```

Quick Actions tippen Sie in einen Kommentar oder die Beschreibung.
Planning Poker: alle schätzen verdeckt, decken gleichzeitig auf.

Element	bündelt	Beispiel
Milestone	Issues bis zu einem Datum	Release v1.0 am 30.11.
Iteration	Issues in festem Zeitraster	Sprint 47, 2 Wochen
Epic	Issues eines großen Vorhabens	Checkout über 3 Sprints

Merksatz: Milestone = Ziel · Iteration = Takt · Epic = Thema.

Epics, Iterations, Roadmaps und Burndown-Charts sind Premium-Funktionen – Milestones gibt es kostenlos.

8 Merge Requests & Code Review

Issue → **Branch** → **Merge Request** → **Review + Pipeline** → **Merge**

- Im Issue **„Create merge request“** → Branch und Draft-MR entstehen
- **Draft:** „noch in Arbeit“ – kann nicht gemergt werden
- **Review:** Kommentar direkt an der Zeile, dann **Approve**
- `Closes #2` in Beschreibung oder Commit schließt das Issue beim Merge
- Kleine MRs werden gründlich gelesen – große nur durchgewunken

```
git fetch && git switch 2-funktion-multiply
# … umsetzen, testen …
git commit -am "feat: multiply() (Closes #2)"
git push
# main hat sich bewegt? Branch aktualisieren:
git fetch && git rebase origin/main
git push --force-with-lease
```

Begriff	bedeutet
Continuous Integration	jeder Push wird gebaut und getestet
Continuous Delivery	jeder grüne Stand ist per Klick auslieferbar
Continuous Deployment	grüne Stände gehen automatisch in Produktion

8 Pipelines mit .gitlab-ci.yml

- **Job:** ein Skript in einem Container, z. B. die Tests
- **Stage:** Gruppe von Jobs – Jobs einer Stage laufen **parallel**
- **Pipeline:** Stages nacheinander; scheitert eine, stoppen die folgenden
- **Runner:** führt die Jobs aus

```
stages: [test]              # Reihenfolge

test:                       # Name des Jobs
  stage: test
  image: python:3.12-slim   # Container
  script:                   # Befehle
    - pip install -r requirements.txt
    - python -m pytest -v
```

Die Datei liegt im Wurzelordner und wird versioniert und reviewt wie Code. Ergebnis: *Build → Pipelines* und im MR.

Abhängigkeiten zwischen Jobs

```
build-image:
  stage: build
  needs: [lint, test]  # wartet nur hierauf
```

`needs: []` startet sofort · `allow_failure: true` darf scheitern · `parallel: matrix` startet denselben Job mehrfach.

Variablen

- **Vordefiniert:** `CI_COMMIT_BRANCH`, `CI_COMMIT_TAG`, `CI_PIPELINE_SOURCE` …
- **Eigene:** *Settings → CI/CD → Variables* – hier gehören Secrets hin
- **Masked:** im Log als `[MASKED]` · **Protected:** nur auf geschützten Branches – für Feature-Branches das Häkchen entfernen

Wann läuft ein Job? rules

```
build-image:
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG
  script:
    - docker build -t calc-service .
```

Trifft eine Bedingung zu, läuft der Job. `when: manual` wartet auf einen Klick.

Cache, Artifacts & Vorlagen

	Cache	Artifacts
Zweck	beschleunigt (Downloads sparen)	übergibt Ergebnisse (Reports, Builds)
garantiert?	nein – darf fehlen	ja

`.python:` (mit Punkt) ist eine Vorlage, Jobs erben mit `extends: .python`. Vor dem Push prüfen: `gitlab-ci-local --list`.

9 Container, Services & Testergebnisse

Dockerfile → **build** → **Image** → **run** → **Container**

- **Dockerfile:** Rezept – Basis, Dateien, Startbefehl
- **Image:** gebackenes, unveränderliches Ergebnis mit Versionsnamen
- **Container:** ein laufendes Image – beliebig oft startbar
- `docker` und `podman`: gleiche Befehle

```
docker build -t calc-service:lokal .
docker run --rm calc-service:lokal
docker images calc-service
```

Images in der Pipeline bauen

```
build-image:
  image: docker:27
  services: [docker:27-dind]  # Docker in Docker
  script:
    - docker build -t calc:$CI_COMMIT_SHORT_SHA .
```

Ohne privilegierten Runner: podman/Buildah oder Kaniko.

Services: Begleit-Container

```
db-check:
  image: postgres:16
  services:
    - name: postgres:16
      alias: db             # Hostname im Job
  script:
    - pg_isready -h db -U calc
```

Der Service startet nur für diesen Job und ist über den `alias` erreichbar.

Testergebnisse im Merge Request

```
test:
  script:
    - python -m pytest --cov=src
        --junitxml=report.xml
  coverage: '/TOTAL.*\s(\d+%)$/'
  artifacts:
    reports:
      junit: report.xml       # MR-Widget
```

Auto DevOps liefert eine fertige Pipeline – gut als Start, aber wenig Kontrolle. In der Praxis: eigene Pipeline plus CI/CD Components.

10 Runner

GitLab verteilt Aufträge, **Runner** erledigen sie: Sie fragen regelmäßig „Gibt es Arbeit für mich?“, führen den Job aus und melden das Ergebnis.

- Instance Runner (für alle), Group Runner, Project Runner
- **Tags** ordnen zu: `tags: [gpu]` läuft nur auf einem Runner mit diesem Tag
- **„pending“** = kein passender Runner frei

Pipeline hängt auf „pending“? Kein Runner aktiv · Job-Tags passen nicht · Runner pausiert oder ausgelastet · geschützter Runner, aber Branch nicht geschützt · gitlab.com: Konto nicht verifiziert.

Executor	So läuft der Job	Einsatz
docker	in einem frischen Container	Standard, isoliert
kubernetes	als Pod im Cluster	skalierbar
shell	direkt auf dem Server	einfach, nicht isoliert
docker-autoscaler	auf Cloud-VMs nach Bedarf	schwankende Last

Eigene Runner für Produktions-Secrets und interne Netze. Betrieb heißt: Updates, alte Images aufräumen, Monitoring.

Release & Ihre Übungen heute

```
git switch main && git pull
git tag -a v1.0.0 -m "Release 1.0.0"
git push origin v1.0.0  # Tag-Pipeline startet
```

Danach *Deploy → Releases → New release* aus dem Tag, Release Notes aus dem Changelog, Milestone verknüpfen und schließen.

Weiterlernen

- Seminar-Repository: Lösungen im Branch `loesungen`, Spickzettel in `nachschlagen/`
- git-scm.com/book – das offizielle Git-Buch
- learngitbranching.js.org – Branches spielerisch üben
- docs.gitlab.com – CI/CD-Referenz

Übung	Arbeitsform
5 · Team-Projekt	Rollen: Maintainer teilt Bildschirm, SSH-Check selbst
6 · Tokens & Keys	Rollen; Token & API jede/r selbst
7 · Sprint-Planung	gemeinsam diktieren, dann eigenes Issue
8 · Pipeline & MRs	Staffellauf, dann Review über Kreuz
9 · Container & Services	Docker selbst, Pipeline gemeinsam
10 · Team-Release	getauschte Rollen, ohne Trainer

Rollen im Team-Projekt: Maintainer (legt an, merged) · Dev A und Dev B (entwickeln, reviewen über Kreuz). Tokens nur per **Direktnachricht**, nie in den Gruppenchat.